

Linear regression and resampling techniques on the Runge function

Using OLS, Ridge, and LASSO regression to fit polynomials

Sander Oppen and Ole Markus Bråthen

University of Oslo

(Dated: October 7, 2025)

Having a good understanding of how to efficiently fit functions to data through regression is vital to many machine learning applications. We explore how OLS, Ridge, and LASSO regression perform on the one-dimensional Runge function, as well as their dependence on hyperparameters. The regression fitting is done through both exact analytical solutions with matrix inversion, and with different gradient descent methods. In addition, we explore resampling techniques to understand the bias-variance tradeoff. Our best performing model was the analytic OLS regression calculated with matrix inversion. The R^2 -score for OLS gives 0.9719, compared to 0.9317 for analytic Ridge regression for the main data set. We find that for smooth one-dimensional low-noise functions like the Runge function, the analytical OLS solution outperforms every other regression and gradient descent algorithm.

I. INTRODUCTION

A good understanding of regression and gradient descent methods is fundamental to all machine learning applications. Having a deep understanding of the benefits and drawbacks of common algorithms in regression makes the process of performing more advanced machine learning easier, which is why we will be studying some of the most common methods in this paper. We are going to use polynomial regression on the Runge function and how we can find an optimal balance between the bias and variance of our model based on its complexity. The Runge function is well suited for this problem, as it has been used as a standard example of a function for which higher polynomial degrees do not always improve the accuracy of the model, mainly due to the increased oscillation close to the edges of the model.

From the Weierstrass approximation theorem, we know that for our Runge function f defined on a closed interval $[a, b] \subseteq \mathbb{R}$, there exists a sequence of polynomials $\{p_n\}$ that converge uniformly to our function. This means that in theory, by choosing a high enough degree for our polynomial, we can ensure any arbitrarily small precision. In practice, when we are interpolating through a finite set of n uniformly spaced points $\{(x_i, y_i)\}$, and therefore we can use a polynomial of degree $< n$. It can be shown that if we construct our sequence of polynomials in this way, where p_n is a polynomial through $n + 1$ uniformly spaced points, then the convergence to f is not uniform, and we therefore have the possibility of the error increasing with higher polynomial degrees. (Runge, 1901, [1]) This is the reason why we want to find alternative approaches to model the Runge function, as normal polynomial interpolation does not produce a good result. When using the regression methods, we also have to balance the model complexity to get the best results, which we will explore through the bias-variance tradeoff.

Section II of the report presents the different methods we will use to study the problem. We present the linear regression models and gradient descent methods we use,

as well as some theory behind the bias-variance tradeoff in OLS models. We then discuss some details behind the implementation of these methods, as well as some verification of the correctness of the implementations. Lastly we discuss briefly how we have used AI tools to help with the production of this report.

We then present the results and discussion of the results in section III. This begins with the simple case of standard ordinary least squares regression, before adding more complex models and comparing the results throughout. After the discussion on regression methods, we explore the bias-variance tradeoff on OLS regression by implementing the bootstrap and cross-validation resampling techniques.

Section IV presents the main findings and interpretations of the study, as well as some limitations of our methods. We present some benefits and drawbacks of the methods and models in the study, as well as some problems for future study.

All implementations and code used for plotting can be found in the project repository.

II. METHODS

A. Regression models

In our study of regression, we will focus on OLS (Ordinary Least Squares), Ridge, and LASSO regression. OLS will serve as a baseline to compare the more complex models with. As it is also the most naive method, we discuss some of its problems that are mitigated with the other models. Ridge and LASSO introduce a hyperparameter, the impact of which we will explore further. We will also compare the results of our LASSO regression with those already implemented in the scikit-learn library.

B. Scaling and splitting of data

We have implemented scaling of the design matrix before we use our regression methods. This has no practical effect for the accuracy or results of OLS regression, but it makes the fits for more complex models and methods significantly better. While the Runge function is such a simple dataset that most of the methods presented in this report could work well without scaling, we get a better understanding of how the methods would work in a more general case with other datasets by scaling the data. We have used the StandardScaler from scikit-learn, which removes the mean and scales to the unit variance for each feature. (Pedregosa et al., 2011, [2]) This scaling method is not ideal if the data contains some outliers, however in our case we know this will not happen, due to the synthetic nature of our dataset. The scaling will also ensure that all the methods are tested with the most even conditions. The Ridge and LASSO regression methods benefit hugely from properly scaled data, whereas the scaling has no effect on the OLS regression. This means that scaling gives us the most fair method for comparison between our models.

The data has also been randomly split into a training and a test batch before running each algorithm. This gives us a dataset to test our models for out-of-sample performance, which is important for determining both the bias and variance of our model. As the splitting is based upon randomness, we have given every program a seed for reproducibility. Due to this randomness, it can also be expected that some results which in theory should match exactly are slightly different from each other. This is mainly due to the difficulty of recreating the identical state for the random number generator across different programs.

C. Gradient descent methods

To improve the performance of our models we will introduce an analysis and discussion of different gradient descent methods. These methods allow us to solve a wider spectrum of problems where we have no explicit formula for the optimal parameters, such as with LASSO regression. We will use two momentum-based gradient descent algorithms, the aptly names Momentum, and ADAM. Momentum uses the previous gradients to influence the size of upcoming steps. This helps to avoid the GD getting stuck in saddle points, where the gradient is near-zero, but we are not at a minimum. An improved version of momentum is given by the ADAM algorithm, which is a complex algorithm that uses exponential decay to get rid of gradients in the far past, as well as correcting for bias in the moments. (Goodfellow et al., p. 305, [3])

AdaGrad and RMSProp are the other two non-momentum based algorithms we will use. AdaGrad adapts the learning rate for the individual model pa-

rameters by scaling based on the sum of all the previous gradients. This results in taking larger steps where the slope in parameter space might not be steep. Some problems of this algorithm is the accumulation of the gradients from the very beginning might lead to the learning rate becoming too small too soon, and thus the algorithm might stop before finding a minimum. RMSProp seeks to fix this by using exponential decay of the historical accumulated gradients, such that gradients from the beginning of the GD do not influence the learning rate after many iterations. (Goodfellow et al., p. 303, [3]) Lastly we use stochastic gradient descent with an adaptive learning rate as a point of comparison to the regular gradient descent.

D. Resampling techniques

We implement the bootstrap resampling technique on OLS regression, and use the results to explore the bias-variance tradeoff for continuous prediction models. In our analysis of the bias-variance tradeoff, we assume that our true data is generated by a model on the form

$$\mathbf{y} = f(\mathbf{x}) + \varepsilon,$$

where ε is the noise, given by a normal distribution with mean 0 and standard deviation σ . When we find the optimal parameters with OLS regression, we are optimizing the cost function

$$C(\mathbf{X}, \boldsymbol{\theta}) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 = \mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})].$$

We can rewrite this cost function to express the bias-variance tradeoff in this model. The bias we define as

$$\text{Bias}[\tilde{\mathbf{y}}] = \mathbb{E}[(\mathbf{y} - \mathbb{E}[\tilde{\mathbf{y}}])^2],$$

and the variance as

$$\text{var}[\tilde{\mathbf{y}}] = \mathbb{E}[(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2]$$

Using these definitions, the cost function can be rewritten.

$$\begin{aligned} \mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] &= \mathbb{E}[(f + \epsilon - \tilde{\mathbf{y}})^2] \\ &= \mathbb{E}[(f - \tilde{\mathbf{y}})^2] + 2\mathbb{E}[\epsilon(f - \tilde{\mathbf{y}})] + \mathbb{E}[\epsilon^2] \\ &= \mathbb{E}[(f - \tilde{\mathbf{y}})^2] + 2\mathbb{E}[\epsilon]\mathbb{E}[f - \tilde{\mathbf{y}}] + \sigma^2 \\ &= \mathbb{E}[(f - \tilde{\mathbf{y}})^2] + \sigma^2 \end{aligned}$$

Here we have used that $\mathbb{E}[\epsilon] = 0$ and $\mathbb{E}[\epsilon^2] = \text{var}[\epsilon] + \mathbb{E}[\epsilon]^2$. We rewrite the new expected value further, using

$$\begin{aligned} \mathbb{E}[(f - \tilde{\mathbf{y}})^2] &= \mathbb{E}[((f - \mathbb{E}[\tilde{\mathbf{y}}]) - (\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}]))^2] \\ &= \mathbb{E}[(f - \mathbb{E}[\tilde{\mathbf{y}}])^2 + (\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2 - 2(f - \mathbb{E}[\tilde{\mathbf{y}}])(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])] \\ &= \text{Bias}[\tilde{\mathbf{y}}] + \text{var}[\tilde{\mathbf{y}}] - 2\mathbb{E}[(f - \mathbb{E}[\tilde{\mathbf{y}}])(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])]. \end{aligned}$$

We show that the remaining expectation value is 0:

$$\begin{aligned} & \mathbb{E}[(f - \mathbb{E}[\tilde{\mathbf{y}}])(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])] \\ &= \mathbb{E}[f\tilde{\mathbf{y}} - f\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}}\mathbb{E}[\tilde{\mathbf{y}}] + \mathbb{E}[\tilde{\mathbf{y}}]^2] \\ &= f\mathbb{E}[\tilde{\mathbf{y}}] - f\mathbb{E}[\tilde{\mathbf{y}}] - \mathbb{E}[\tilde{\mathbf{y}}]^2 + \mathbb{E}[\tilde{\mathbf{y}}]^2 = 0. \end{aligned}$$

Therefore

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \text{Bias}[\tilde{\mathbf{y}}] + \text{var}[\tilde{\mathbf{y}}] + \sigma^2.$$

There are a few interesting observations to make from this result. Firstly, if we manage to get a fit with zero bias and variance, we will still expect there to be some error due to the irreducible noise term σ^2 . The bias term measures how far our model is on average from the true values given by $f(x)$. A high bias is often a sign of underfitting, as our model is not able to accurately predict any values correctly. The variance term measures how much our model would change if we were to train it on a different sample from the same dataset. If this term is large, we might be overfitting, as the model has become too specialized to the specific set of data, and it generalizes poorly. Finding a good balance between these two terms is crucial in finding a well-fitting model. Even though we might have two models with the same MSE, one might be overfitted with high bias and low variance, while one might be underfitted. This means that we should not only look at the MSE of our models to determine their performance, but also at the bias and variance terms to ensure we have a good balance between the two.

Lastly, we look at the k -fold cross-validation method for resampling, and look at its results in comparison with the bootstrap method. We use the scikit-learn implementation of k -fold cross validation, as this method is well established and will make a good comparison for our simple bootstrap method.

E. Implementation

All methods are implemented as functions, apart from gradient descent, which is implemented as a class. The class structure allows us to change the gradient function easily without any reused code. The analytical solutions also use the pseudoinverse instead of the matrix inverse, as this is more numerically stable and solves more cases where there might be no exact solution. All methods are contained in a `util.py` file, and we import the methods to Jupyter notebooks for each section of the results. All instances where randomness is used in the code, the randomness is seeded to give consistent results. We have also implemented some simple test functions in the file `util_test.py` to verify our closed-form solutions against the scikit-learn library.

F. Use of AI tools

We have used AI tools to aid in the writing of this project. None of the text is directly generated from the LLM, but it has been reviewed and improved upon using suggestions from the AI. The AI has also been used for code debugging and LaTeX help. We have tried to not use an excess amount of AI, as the climate impact of large language models is not insignificant. For our AI, we have used the locally hosted GPT UiO, which uses models from OpenAI. We have used this model as it is the most privacy-focused option available. All conversations with the AI can be found in the project repository

III. RESULTS AND DISCUSSION

A. Ordinary Least Squares

We begin by looking at the simplest form of linear regression, the closed form solution to the ordinary least squares method, given by

$$(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

In figure 1, we have plotted the MSE (mean squared error) and R^2 scores of this regression as a function of the polynomial degree.

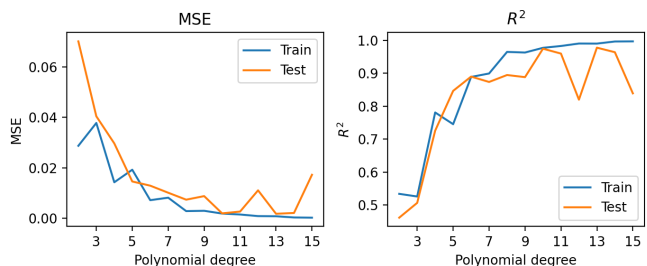


Figure 1. MSE and R^2 score as a function of the polynomial degree

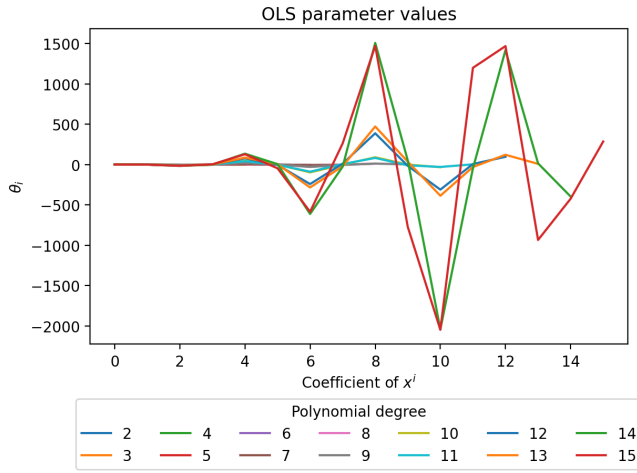


Figure 2. OLS parameter values as a function of the polynomial degree

Noise factor	MSE	R^2
0	0.0020	0.9720
0.01	0.0017	0.9719
0.1	0.0121	0.8860
1	1.1143	-0.3218
10	72.8206	-0.1491
100	8490.5448	-0.0652

Table I. MSE and R^2 for different noise levels ($c \cdot N(0, 1)$) on OLS regression ($n = 100$, degree 10)

We observe that there is clearly a trend towards the higher degree polynomials being better fits for the Runge function. Towards the higher polynomial degrees, we begin to see that the R^2 score for the training data continues to improve gradually, while the testing data shows worsening performance. Figure 2 gives us a clearer view into what is happening with the higher polynomial degrees. The magnitude of the coefficients for higher degree polynomials increases tremendously, which suggests that we are in a stage of overfitting the model to the training data, and we therefore see a gap between the models performance on the training- and testing data. There are several ways of dealing with the problem of large coefficients, and we will continue with exploring Ridge regression. Finally, it is important to note the effect of noise on our model. We have used a normal distribution multiplied by a constant factor c to add noise to the function. Table II shows how increasing the constant noise factor quickly ruins the accuracy of our fits. At a factor of 1, we have an R^2 -score of -0.32 , which means our OLS model fails to pick up any trend in the data.

B. Ridge regression

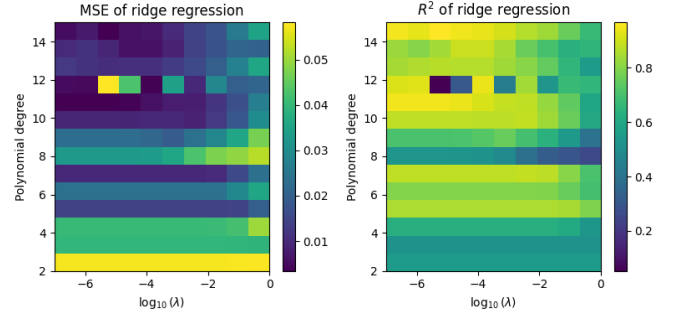


Figure 3. MSE and R^2 score for ridge regression, as a function of polynomial degree and hyperparameter λ

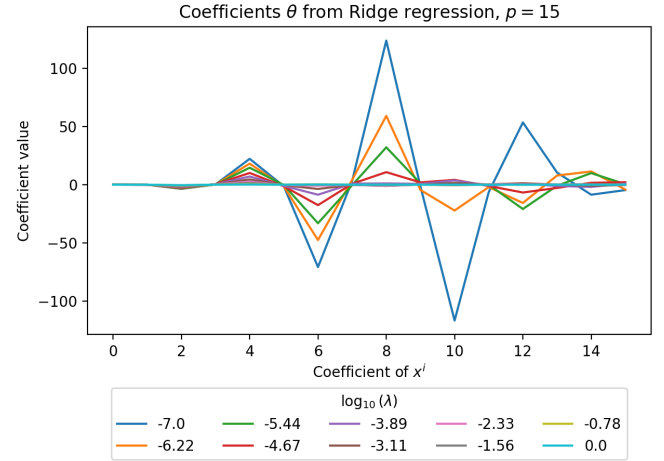


Figure 4. Coefficients obtained from Ridge regression with a degree 15 polynomial, for different λ values

Figure 3 shows the accuracy of Ridge regression on the test sample with the same data as for the OLS regression. For the smallest value of $\lambda = 10^{-7}$, the results are almost identical to the OLS accuracy. This is to be expected, as the cost function for Ridge regression with $\lambda = 0$ is the same as the OLS cost function. There are some outliers in the results for polynomial degree of 12, however these are most likely numerical errors, as we are still relying on inverting matrices for finding the regression parameters.

There is also a clear trend towards greater values of λ leading to a worse fit. This is likely due to the coefficients being too constrained to accurately recreate the curvature of the Runge function, which becomes clear when looking at figure 4. There are still some fluctuations in the parameter values for the lower values of λ , however even for the lowest values there is a decrease of an order of magnitude in the parameter values. The more constrained coefficients suggests that the Ridge regression does not overfit to the same degree as the OLS

regression, however we have to be careful about not constraining the coefficients to such a degree as to produce an underfitted model.

Noise factor	MSE	R^2
0	0.0049	0.9317
0.01	0.0053	0.9108
0.1	0.0144	0.8643
1	1.0590	-0.2562
10	72.1175	-0.1380
100	8266.9630	-0.0372

Table II. MSE and R^2 for different noise levels ($c \cdot N(0, 1)$) on Ridge regression ($n = 100$, degree 10)

Table

C. Gradient descent

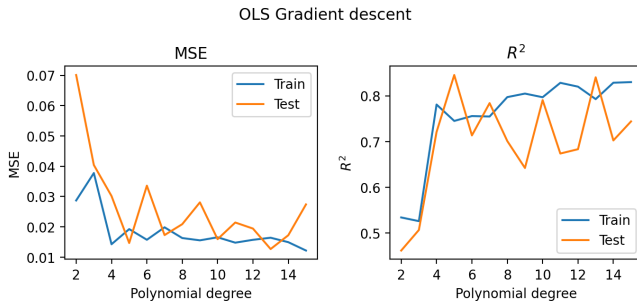


Figure 5. MSE and R^2 for OLS using gradient descent

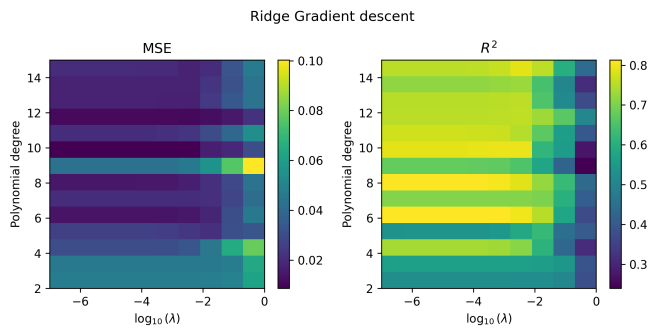


Figure 6. MSE and R^2 of Ridge regression using gradient descent

We begin by checking the effects of gradient descent against the results of the previous sections, with a fixed learning rate of 0.5. For OLS regression, as seen in figure 5, the results are almost exactly the same as the analytical solutions, apart from the fit being slightly worse, as indicated through the lower R^2 score. There seems to be slightly more irregularity with the fits using gradient descent, which most likely has to do with the fixed learning rate converging better for certain polynomial degrees.

Figure 6 shows some major improvements from the analytical solution for Ridge regression. There seem to be no major numerical errors causing outliers, however this has come at the expense of some accuracy. As with the OLS regression, the R^2 score is slightly lower, even though the MSE of both match closely with the analytical solutions.

Learning rate	OLS	Ridge
0.05	0.7750	0.7745
0.1	0.8055	0.8047
0.15	0.8169	0.8159
0.17	$-7.97 \cdot 10^{14}$	$-8.52 \cdot 10^{14}$

Table III. R^2 score for given fixed learning rates, for OLS and Ridge regression (degree 12, $\lambda = 10^{-4}$)

The issue with fixed learning rates is illustrated in table III. We see that there is a trend towards better fits with higher learning rates, but picking a learning rate too high will cause the gradient descent method to diverge. Even in scenarios where the gradient descent seemingly converges to a reasonable value, we might have some oscillation around the true minimum, due to the steps the gradient descent takes constantly overshooting the minimum, and skips to the other side of the convex "bowl". As the area of guaranteed convergence with a fixed learning rate is determined by the hessian matrix of the gradient, changing the hyperparameter λ will also have a large effect on the range of learning rates that converge.

The greatest problem with the fixed learning rates is that every unique configuration of polynomial degree and hyperparameters might require lots of work to find the unique optimum. This might work fine in this case where we have a specific setup, but if we wish to find a more general best fit which might require checking many combinations of degrees and hyperparameters, the amount of work quickly becomes huge.

D. Adaptive learning rates

Parameter	Value
Initial & fixed learning rate	$\epsilon = 0.05$
Momentum	$\alpha = 0.995$
RMSProp	$\rho = 0.999$
ADAM	$\rho_1 = 0.9, \quad \rho_2 = 0.999$

Table IV. Parameters used for gradient descent methods

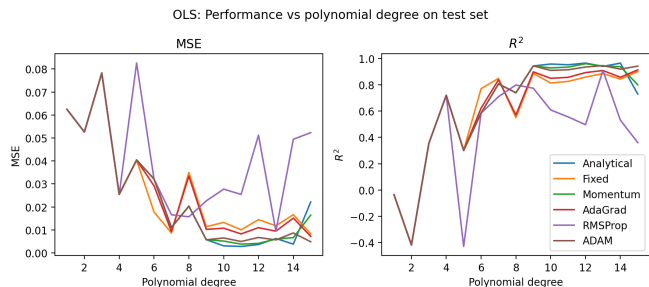


Figure 7. MSE and R^2 for OLS with adaptive learning rates against polynomial degree, on test sample.

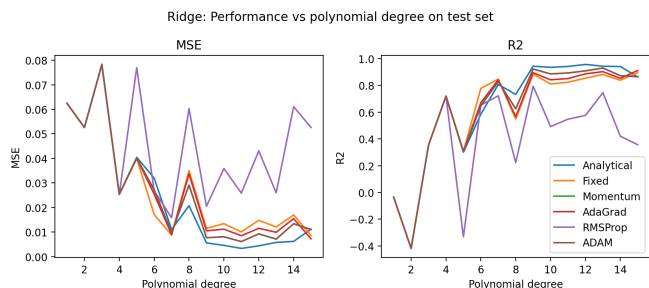


Figure 8. MSE and R^2 for Ridge ($\lambda = 10^{-4}$) with adaptive learning rates against polynomial degree, on test sample.

We continue working with the 12th degree polynomial and the hyperparameter $\lambda = 10^{-4}$ to see how methods to update the learning rate can be used to get a more accurate fit. We will only be looking at the model performance on the test data, to simplify the analysis of the results. Figure 7 shows how the different methods perform on differing polynomial degrees for the test sample for a small dataset ($n = 51$). It is immediately noteworthy that RMSProp is significantly more unstable than all the other models. This is interesting, as in theory, RMSProp is an improvement upon the AdaGrad algorithm, where the only difference between the two is the exponential decay based upon the parameter ρ for RMSProp. This decay is intended to make RMSProp act like an instance of AdaGrad when it finds a convex bowl in the gradient function, but in practice it is possible that it simply finds a worse convex bowl to optimize within, compared to the bowl AdaGrad finds. (Goodfellow et al., p. 304, [3])

Adam and momentum are both momentum-based methods, and in this instance they have performed seemingly identically. These methods are also the most stable in terms of fluctuations away from the analytical solutions, and seem the most ideal for our dataset. It is also interesting to note that for some degrees, the adaptive learning rates produce better fits than the "exact" analytic solution provides. In theory, the exact solution is always optimal, however due to numerical errors in the matrix inversion, we might expect that the analytical solution can be improved upon.

The same regression is done with Ridge regression with hyperparameter $\lambda = 10^{-4}$ is done in figure 8. Again RMSProp is vastly more unstable than the other methods, with it only sporadically being able to match the performance of the other methods. This might suggest that the poor performance of RMSProp is not caused by the gradient function it optimizes, but rather the dataset. To further show how poorly RMSProp performs compared to the other methods, we can repeat the same process used to create figure 6, only replacing the fixed rate gradient descent with RMSProp. The results of this experiment is shown in figure 9

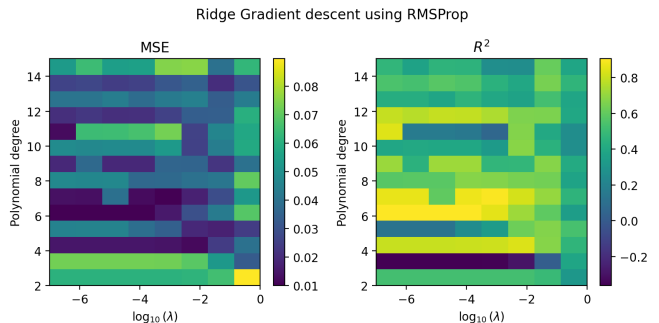


Figure 9. Ridge Gradient descent with RMSProp

With the fixed learning rate gradient descent, there is a clear trend with the higher polynomial degrees and smaller λ values producing better fits. This trend is not clear when using RMSProp, as it is seemingly somewhat random which polynomial degrees and λ values produce good fits. While further tests are needed to determine precisely what is happening here, a reasonable assumption might be that the RMSProp algorithm is too aggressive in trying to find different minima in the gradient than AdaGrad, and it therefore skips over some more ideal solutions.

E. Lasso regression

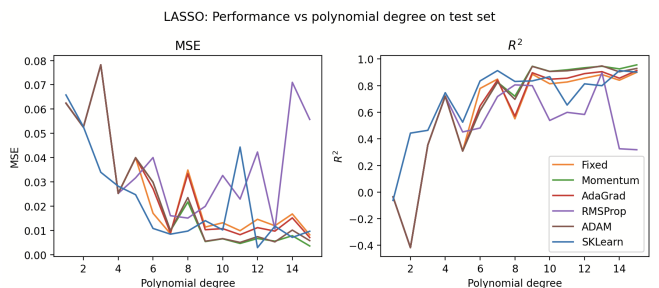


Figure 10. MSE and R^2 for LASSO ($\lambda = 10^{-4}$) with adaptive learning rates against polynomial degree, on test sample.

As the LASSO gradient has no closed-form analytical solution, we do not have a baseline to compare the gra-

dient descent methods with. As a substitute, we have used the implementation of LASSO regression given by scikit-learn, with the maximum number of iterations set to 1000, to match our stopping criterion. We again see that the momentum-based methods consistently performs better than the rest by a good margin in every polynomial degree.

The RMSProp results also shows large fluctuations in performance, as with both OLS and Ridge. However, the stability and accuracy seems somewhat improved, as both the MSE and R^2 scores of the largest fluctuations have decreased in amplitude. This is especially noticeable with the lower degree polynomials. For both OLS and Ridge regression, fitting a 5th degree polynomial with RMSProp would yield a negative R^2 score, but in the case of LASSO regression with RMSProp, the R^2 score beats all other methods. As RMSProp has shown that it is quite irregular in the results it gives, it is hard to know whether the apparent improvements are due to the LASSO gradient being better suited for RMSProp gradient descent, or whether we have gotten lucky with the choices of hyperparameters.

Some of the gradient descent methods also seem to beat the scikit-learn implementation in terms of the fit accuracy, especially with the higher degree polynomials. Looking at the MSE for the different methods, it is clear that the results produced by scikit-learn are more consistent and follow a smoother curve, whereas our implementations have much higher variations based on the polynomial degrees. Our momentum-based models also display many of the same tendencies of a smooth MSE curve, as well as high fit accuracy, especially for higher degree fits.

F. Stochastic gradient descent

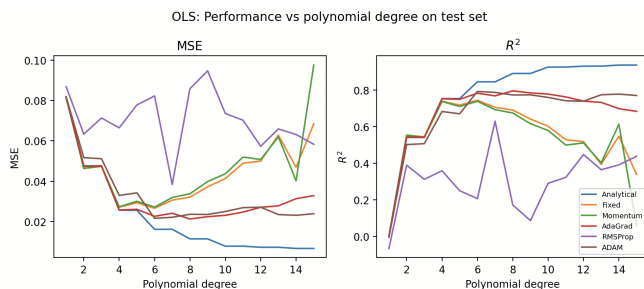


Figure 11. MSE and R^2 for OLS using stochastic gradient descent, on test sample ($n = 500$).

Parameter	Value
Epochs	50
Minibatch size	5
Hyperparameter	$\lambda = 10^{-4}$
Initial & fixed learning rate	$\epsilon = 0.05$
Momentum	$\alpha = 0.06$
RMSProp	$\rho = 0.99$
ADAM	$\rho_1 = 0.9, \rho_2 = 0.999$

Table V. Parameters used for stochastic gradient descent

To perform stochastic gradient descent, we have changed the number of data points to $n = 500$, as we found that using the previous $n = 50$ gives irregular and inconsistent answers. It is again worth noting that RMSProp is clearly the worst performing out of our methods, which is almost to be expected from the previous tests. We seem to get a divergence between the other methods for polynomials of degree 5 and higher. For the momentum gradient descent, we have had to set the momentum parameter α to such a low number due to the results diverging for any higher values.

This is not to be expected, as a more reasonable α value would be somewhere around 0.9, like in the previous examples. Due to the small momentum parameter, it is to be expected that momentum and fixed learning rate GD will perform equally, as if we set $\alpha = 0$, we get the fixed learning rate algorithm. From the previous sections, it is perfectly reasonable to assume that momentum GD should perform equally as ADAM, as they are both momentum-based. The discrepancy between these two models means that there is either a problem with our implementation of Momentum, or that the gradient is unsuited for momentum stochastic GD in some way.

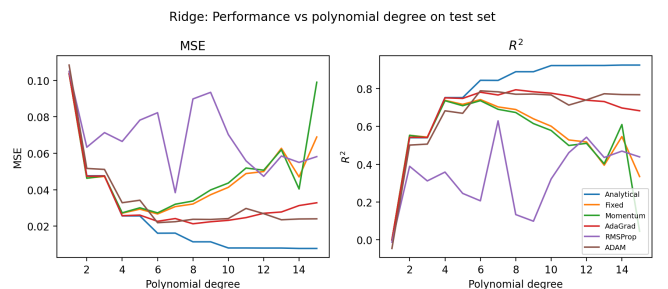


Figure 12. MSE and R^2 for Ridge using stochastic gradient descent, on test sample ($n = 500$).

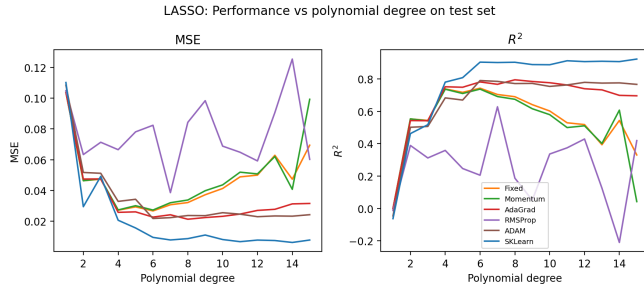


Figure 13. MSE and R^2 for Ridge using stochastic gradient descent, on test sample ($n = 500$).

When using the stochastic gradient descent methods on Ridge and LASSO regression, we get results that are extremely similar to each other, much different from when we used normal GD methods. From the plots it seems that stochastic GD finds nearly the same parameters regardless of which gradient we use. This is quite plausible, as much of the point behind SGD is that using small random samples will speed up convergence, which might be what is happening here.

For our purposes, there does not seem to be any reason to use stochastic gradient descent over normal GD methods. As our dataset is not too large, and we get extremely good fits in a reasonable runtime with normal GD methods, stochastic GD would only serve to introduce more randomness and less accuracy in the regression. If it were necessary to use the stochastic methods due to having a much larger dataset, then it is clear that ADAM still produces the most consistent and stable results out of all the methods we have tested.

G. Bias variance tradeoff and resampling techniques

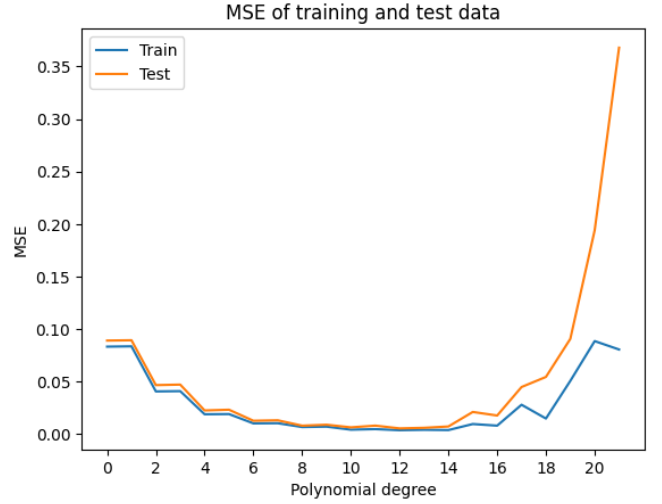


Figure 14. MSE of training and test data, as a function of polynomial degree fitted.

Figure 14 shows the error of regular OLS regression on the Runge function, for different polynomial degrees. It provides a good insight into where we might be over- and underfitting our models. For the low polynomial degrees (≤ 3), the model performs badly on both the training and test data, suggesting that it is failing to accurately predict any of the data in the dataset. For high model complexity (degree ≥ 16), we see the training and test error diverging, suggesting overfitting. The model has fitted too much to the training data, and it is therefore picking up trends which are simply statistical noise.

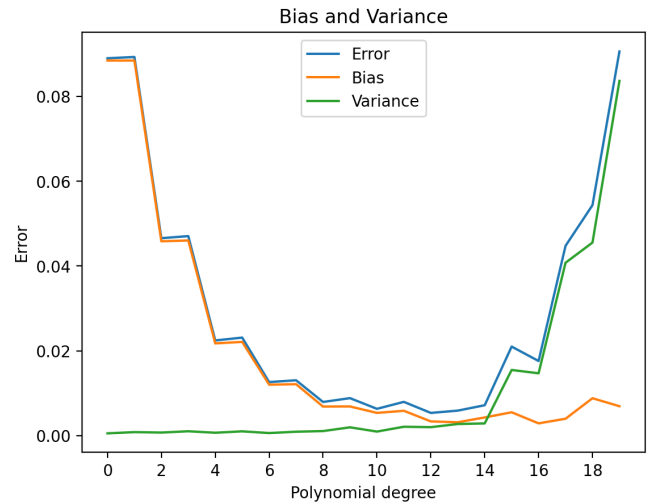


Figure 15. Bias and Variance as function of polynomial degree, with bootstrap resampling on OLS

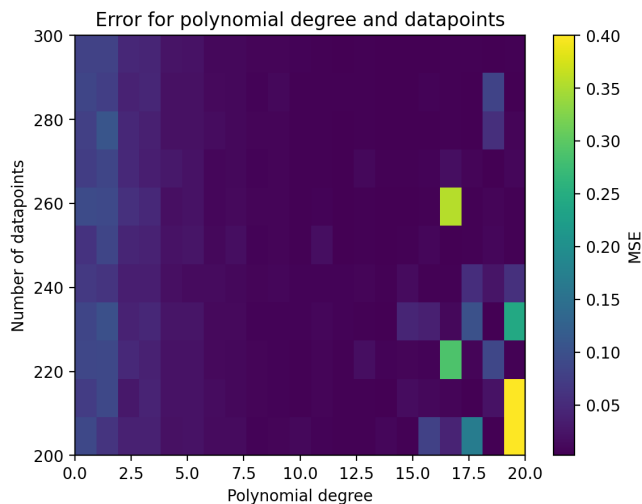


Figure 16. OLS error as a function of data points and polynomial degree, capped at 0.4.

The bias-variance analysis of the Runge function we have done can be seen in figure 15. We have used $n = 200$ data points, and 1000 bootstraps for each polynomial degree. The error, bias, and variance are all calculated on the testing data set, and so we see that it matches with the test error in figure 14. For the lower degree models, we have almost no variance, but a high degree of bias.

This is to be expected, as the models are underfitted for low polynomial degrees. The underfitting leads to the models not picking up on any trends in the data, and they are therefore low variance as a result. For high polynomial degrees, we have the same level of total error, however the variance term has taken over, with little to no bias left. This is due to overfitting, where we get low bias due to the models fitting the data too well, and high variance as a result. To achieve an overall good fit, we should in this instance pick a polynomial with degree around $8 \leq p \leq 14$, as this is the area with both low variance and bias.

It is important to note that this range only works for this specific number of data points. To illustrate this, we can look at the total error as we vary both the degree and the number of data points. Figure 16 shows the error for OLS regression as a function of model complexity and data points. It is important to note that the maximum error shown is capped at 0.4, although some error terms are higher. This is to better illustrate the general trend, rather than just the outliers.

We see that there is the same trend as in figure 15, where polynomial degrees that are low and high produce greater error levels. There is also a trend towards the error decreasing for the high degree models with more data points. This is caused by the variance decreasing with more data points, as with more data points, the variation from sample to sample of the dataset decreases.

H. Cross validation as resampling techniques

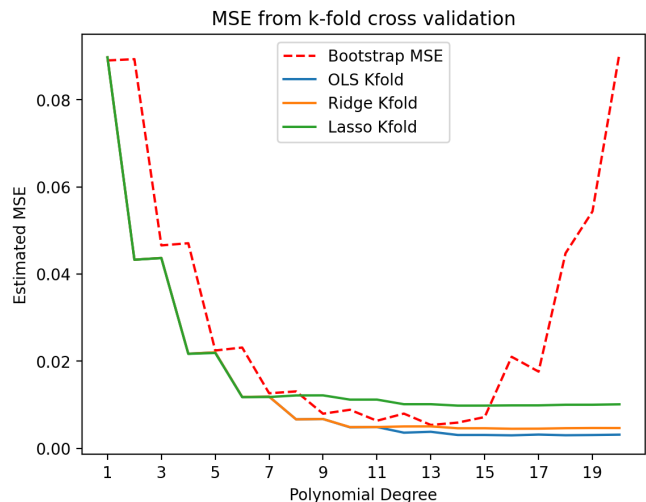


Figure 17. k -fold cross validation using OLS, Ridge and Lasso, compared to the Bootstrap MSE from section III G

Parameter	Value
Folds	$k = 5$
Hyperparameters	$\lambda = 10^{-4}$

Table VI. Parameters used for k -fold cross validation

We have used the scikit-learn implementation of k -fold cross validation, and therefore also used the scikit-learn methods for OLS, Ridge, and LASSO regression. The stopping parameters for these methods have been adjusted to match the parameters we have implemented.

It is immediately clear that k -fold cross validation does not suffer from the same problems as bootstrap when it comes to the error increasing with higher polynomial degrees.

This is probably not because k -fold cross validation inherently prevents overfitting, but rather because it may provide a more reliable and stable estimate of the model's generalization error. In cross validation, each fold acts as an independent validation set, and the mean squared error (MSE) is averaged across these folds, which tends to smooth out fluctuations that can occur in individual overfitted fits. This may explain why the k -fold MSE curves remain stable even at high polynomial degrees, while the bootstrap estimates on the same data show a clear increase in MSE beyond approximately degree 15.

A likely reason for this difference is the structure of the dataset itself: we use a very large number of evenly spaced and deterministic x -values, resulting in nearly identical distributions across all folds. This makes the training and validation subsets in k -fold cross validation highly similar, allowing even high-degree polynomials to

generalize well within the data range.

In contrast, the bootstrap method resamples with replacement, which introduces more variation in the training subsets and amplifies overfitting effects, leading to the observed rise in MSE. Combined with the relatively low noise level in the data, this may explain why all three k -fold methods—OLS, Ridge, and LASSO—show consistently low and stable MSE values even for high polynomial degrees.

IV. CONCLUSION

We have found that for the Runge function with evenly spaced points, analytical OLS regression produces the best fit, as compared to more complex models. This is almost to be expected in a synthetic dataset like the one we have worked with, which is too simple for showing the flaws in OLS regression. On the more complex models, we see that LASSO can match and sometimes beat the performance of Ridge regression, but that this depends heavily on the gradient descent method used.

There is a large difference in the performance of the different gradient descent methods. Fixed learning rate GD can perform well if the learning rate is chosen well, but we also see that small changes in the fixed learning rate can lead to large performance gains or losses. Momentum-based methods are much more stable and give better re-

sults for all regression methods on the Runge function. RMSProp is unable to match any of the other gradient descent methods in terms of accuracy or stability, and it is therefore not recommended for this application. Further study is required to determine why this instability occurs with this particular dataset.

When using resampling methods, k -fold cross validation provides a more stable generalization of estimates. Bootstrap resampling provides a clear view into cases of overfitting, mainly due to the variability of the sampling. In only looking at a smooth one-dimensional, low-noise function like the Runge function, it is to be expected that simple models for regression beat the more complex ones. The Runge function might provide an optimization landscape that is too smooth for any complex regression or gradient descent models to compete with simple analytical solutions. If this study were to be repeated with a more complex function we might expect to see other methods perform better. The parameters we have chosen are also quite arbitrary, and one might find better results by optimizing each method using a grid search for hyperparameters. One avenue for further study is to look at how using the Chebyshev points instead of uniformly spaced points could impact the regression accuracy. These points are designed to get a convergent series of polynomials using polynomial interpolation, so they might improve the performance of the linear regression models.

-
- [1] C. Runge *et al.*, Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten, *Zeitschrift für Mathematik und Physik* **46**, 20 (1901).
 - [2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cour-
 - napeau, M. Brucher, M. Perrot, and E. Duchesnay, Scikit-learn: Machine learning in Python, *Journal of Machine Learning Research* **12**, 2825 (2011).
 - [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, 2016) <http://www.deeplearningbook.org>.